

ControlNet: Implementation Report

1 Introduction

This report describes my end-to-end implementation of ControlNet [1], a neural network architecture that adds spatial conditioning controls to large pretrained text-to-image diffusion models. I implemented the core components from scratch following the paper and the official source code (`cldm/cldm.py`), trained on the authors’ own tutorial dataset (Fill50K), and evaluated using standard metrics.

The goal was not to use an existing ControlNet library, but to implement the architecture myself and demonstrate understanding of the key concepts: zero convolutions, the trainable encoder copy, conditioning injection, and the training procedure.

2 Background

ControlNet addresses a fundamental limitation of text-to-image diffusion models: while models like Stable Diffusion can generate high-quality images from text prompts, they offer limited control over spatial composition. ControlNet solves this by learning to inject spatial conditioning (edges, depth maps, poses, etc.) into a frozen pretrained model without destroying its learned capabilities.

The key challenge is that finetuning a large pretrained model on a small conditioning dataset risks catastrophic forgetting. ControlNet avoids this through two mechanisms: (1) freezing the original model entirely, and (2) connecting a trainable copy through zero-initialized convolution layers that ensure no harmful noise enters the pretrained network at the start of training.

3 Architecture

My implementation follows the official source code. The architecture consists of four components:

3.1 Zero Convolutions

These are 1×1 convolution layers with both weight and bias initialized to zero. At the start of training, they output exactly zero, so the pretrained model is completely undisturbed:

$$y_c = F(x; \Theta) + \mathcal{Z}(F(x + \mathcal{Z}(c; \Theta_{z1}); \Theta_c); \Theta_{z2}) \quad (1)$$

Since $\mathcal{Z}(\cdot; \cdot) = 0$ initially, we have $y_c = F(x; \Theta) = y$, preserving the original model output. As training progresses, the zero convolution weights grow from zero, gradually introducing the ControlNet’s influence.

3.2 Trainable Encoder Copy

I deep-copy the 12 encoder blocks and 1 middle block from SD’s UNet. All weights start as exact copies of the pretrained model, so no layer is trained from scratch. The paper emphasizes this point: the trainable copy begins as a fully functional feature extractor, serving as a strong backbone for learning conditional controls.

3.3 Conditioning Embedding $E(\cdot)$

This small convolutional network converts a 512×512 pixel-space conditioning image into a 64×64 feature map matching the latent resolution. The paper text describes 4 convolution layers with 4×4 kernels and 2×2 strides using 16, 32, 64, 128 channels. However, the actual source code (`input_hint_block` in `cldm.py`) uses 8 convolution layers with 3×3 kernels:

$$3 \rightarrow 16 \rightarrow 16 \rightarrow 32 \downarrow \rightarrow 32 \rightarrow 96 \downarrow \rightarrow 96 \rightarrow 256 \downarrow \rightarrow 320_{\text{zero}}$$

I followed the source code implementation, not the paper text. The final layer is a zero-initialized convolution, providing an additional safety mechanism.

3.4 Skip Connection Injection

The 13 zero-conv outputs (12 from the encoder copy + 1 from the middle block) are added to the frozen UNet decoder’s skip connections. The frozen decoder is never modified. This is computationally efficient since the frozen encoder requires no gradient storage.

4 Training

4.1 Setup

Parameter	Value
Base model	Stable Diffusion 1.5 (frozen)
Dataset	Fill50K (49K train / 1K val)
Loss	MSE on predicted noise (Eq. 5 in paper)
Optimizer	AdamW, $\text{lr} = 1 \times 10^{-5}$
Prompt dropout	50% replaced with empty string
<code>sd_locked</code>	True (decoder frozen)
Mixed precision	fp16
Hardware	NVIDIA A100 40GB (Colab Pro)

Table 1: Training configuration, following the paper’s defaults.

4.2 Loss Function

The training objective is the standard diffusion denoising loss (Eq. 5 from the paper):

$$\mathcal{L} = \mathbb{E}_{z_0, t, c_t, c_f, \epsilon \sim \mathcal{N}(0,1)} [\|\epsilon - \epsilon_\theta(z_t, t, c_t, c_f)\|_2^2] \quad (2)$$

where z_t is the noisy latent at timestep t , c_t is the text prompt embedding, and c_f is the conditioning image feature from the hint encoder. No custom losses are needed.

4.3 Prompt Dropout

During training, I randomly replace 50% of text prompts with empty strings. The paper explains this increases the ControlNet’s ability to directly recognize semantics in the conditioning image as a replacement for the prompt. This is critical for the model to learn spatial control that works even without text guidance.

4.4 Dataset: Fill50K

Fill50K is the paper’s tutorial dataset consisting of 50,000 image pairs:

- **Source (condition):** Circle outlines on black backgrounds
- **Target:** Filled circles with colored backgrounds
- **Prompts:** Descriptions like “cyan circle with brown background”

Source images are normalized to $[0, 1]$ and target images to $[-1, 1]$, following the official `tutorial_dataset.py`.

5 Two Training Runs

I ran two versions that differ in how many parameters are trainable, stemming from a subtle implementation detail.

5.1 V1: Frozen Encoder Copy (12.6M trainable)

In my first run, I froze the SD UNet before creating the ControlNet copy. Because `copy.deepcopy` preserves the `requires_grad` flag, the deep-copied encoder blocks inherited `requires_grad=False`. Only the freshly created zero convolutions and conditioning encoder were trainable (12.6M out of 361M total parameters).

- Batch size: 4, Steps: 50,000 (~ 4 epochs)
- Training time: 4.83 hours
- VRAM usage: ~ 12.5 GB

5.2 V2: Unfrozen Encoder Copy (361M trainable)

I fixed this by explicitly setting `requires_grad=True` for all ControlNet parameters after construction. This matches the paper’s intention where the full encoder copy is trainable.

- Batch size: 4, Steps: 25,000 (~ 2 epochs)
- Training time: 2.77 hours
- VRAM usage: ~ 17 – 18 GB

6 Results

6.1 Quantitative Comparison

Metric	V1 (12.6M, 50K steps)	V2 (361M, 25K steps)	Better
FID $\downarrow$	119.81	134.11	V1
PSNR (dB) $\uparrow$	8.61	8.50	V1
CLIP Score $\uparrow$	0.326	0.3275	V2
Final loss	0.0044	0.0040	V2
Zero-conv norm	3.24	1.44	–
Training time	4.83h	2.77h	V2

Table 2: Comparison of V1 and V2 training runs.

Note on paper baselines: The paper reports CLIP Score ~ 0.26 and FID ~ 15.27 , but these are on ADE20K semantic segmentation, a much harder and more diverse task. Fill50K is a toy dataset with no published baselines, so direct comparison to the paper’s numbers is not applicable. My CLIP scores being higher than the paper’s is expected since Fill50K prompts are much simpler than ADE20K descriptions.

6.2 Visual Quality

V1 produces cleaner outputs overall. The generated circles have correct positions and sizes with approximate color matching. Backgrounds have some texture from SD’s artistic interpretation but are generally clean.

V2 shows more artifacts. While circles are correctly positioned, backgrounds are noisier with visible texture artifacts, and some samples exhibit line patterns. This is attributable to encoder drift and insufficient training steps.

6.3 Why V1 Outperformed V2

This was initially surprising since V2 is the architecturally correct implementation per the paper. Three factors explain the result:

1. **V2 trained for half the steps.** 25,000 steps with 361M trainable parameters is insufficient for the full encoder to converge. The paper’s experiments use significantly more training on more complex tasks.
2. **Encoder drift.** With the full encoder trainable, the copied weights diverge from the pretrained SD initialization, degrading learned feature quality. The lower zero-conv norms in V2 (1.44 vs 3.24) support this: the encoder absorbs more adaptation work, but at the cost of feature quality.
3. **Fill50K is too simple.** For a trivially easy conditioning task, 12.6M trainable parameters (zero convolutions + hint encoder) are sufficient. The full encoder capacity adds optimization noise without benefit.

For more complex conditioning tasks (Canny edges, depth maps, human poses), the full encoder training would be necessary, as the paper demonstrates.

7 Convergence Behavior

Both runs exhibit the convergence phenomenon described in the paper (Section 3.3, Figure 4), where the model transitions from ignoring conditioning to following it:

- **Steps 100–500:** Generated images are random SD outputs with no circles. Zero convolutions output ~ 0 , so ControlNet has no influence.
- **Steps 1000–3000:** Circular shapes begin appearing. Zero-conv weights have grown enough to transmit spatial signal.
- **Steps 4000–6000:** Clear circles with colored backgrounds. Conditioning is followed consistently.
- **Steps 10000+:** Refined and stable outputs.

The transition occurs around steps 2000–5000, earlier than the paper’s reported 6K–10K range. This is expected because Fill50K is much simpler than the Canny edge and depth map tasks the paper evaluated on. The convergence is also more gradual than the dramatic “sudden snap” shown in Figure 4, likely because filling circles with solid colors is a trivially easy task for Stable Diffusion.

The zero-conv weight norms provide direct evidence of this process. They start at exactly 0.0 (by construction) and grow throughout training: to 3.24 in V1 and 1.44 in V2. This growth represents the ControlNet progressively “turning on” its influence over the frozen decoder.

8 Evaluation Methodology

I evaluated using three metrics:

FID (Frechet Inception Distance) measures distributional distance between generated and target image sets. Lower is better. Computed using `clean-fid` on 100 generated/target pairs.

CLIP Score measures text-image alignment via cosine similarity between CLIP embeddings of the generated image and its text prompt. Higher is better. Computed on 50 samples using ViT-B-32 from OpenCLIP.

PSNR (Peak Signal-to-Noise Ratio) measures pixel-level reconstruction fidelity. Higher is better. Note that PSNR is inherently limited here because Stable Diffusion generates images with its own artistic interpretation, so pixel-perfect reconstruction is not expected.

9 Inference

I use DDIM sampling with 50 steps and Classifier-Free Guidance (CFG) at scale 9.0. Following Section 3.4 of the paper, ControlNet conditioning is applied to both the conditional and unconditional passes:

$$\epsilon_{\text{pred}} = \epsilon_{\text{uc}} + \beta_{\text{cfg}} \cdot (\epsilon_c - \epsilon_{\text{uc}}) \quad (3)$$

where both ϵ_c and ϵ_{uc} receive the conditioning image through ControlNet.

The paper also describes an optional CFG Resolution Weighting scheme ($w_i = 64/h_i$) for edge cases with no text prompt. I did not implement this since all Fill50K samples have prompts and the official source code defaults to uniform scaling (`self.control_scales = [1.0] * 13`).

10 Key Takeaways

1. **Zero convolutions are elegant.** Initializing connection layers to zero so the pretrained model is undisturbed at step 0 is simple but powerful. It eliminates harmful noise during early training and allows conditioning to be introduced gradually.
2. **Fill50K convergence is real but gentle.** The transition from uncontrolled to controlled generation is visible in training progression images, happening around steps 2000–5000. It is less dramatic than the paper’s Figure 4 because circles are a simpler conditioning signal than Canny edges on natural images.
3. **More trainable parameters is not always better.** For simple tasks, training only the zero convolutions and hint encoder can outperform full encoder training, especially with limited training time. The pretrained encoder features are strong enough for easy conditioning.
4. **The architecture is efficient.** Training uses 12–18 GB VRAM on a single GPU and completes in a few hours. The paper’s claim of $\sim 23\%$ more memory and $\sim 34\%$ more time compared to base SD training holds up in practice.

References

- [1] Zhang, L., Rao, A., & Agrawala, M. (2023). Adding Conditional Control to Text-to-Image Diffusion Models. *ICCV 2023*. arXiv:2302.05543.